QALight

Frontend-розробка

Заняття 19. Замикання

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Замикання.
Функція зворотного виклику. Функції вищого порядку.
Робота з функціями map, filter та reduce.

Замикання

Функція зворотного виклику. Функції вищого порядку.

Функціональне програмування

Теорія: Функціональне програмування

Функціональне програмування - це **парадигма** програмування, стиль побудови структури та елементів програм, який трактує обчислення як оцінку математичних функцій і уникає зміни стану та змінних даних. Функціональне програмування дозволяє розробникам писати код на основі поставленого завдання та структурувати його відповідно до функціонального підходу, спрощеного для читання.

Переваги функціонального програмування

- допомагає ефективно вирішувати проблеми більш простим способом.
- покращує модульність.
- дозволяє реалізувати лямбда-числення в нашій програмі для вирішення складних задач.

Недоліки функціонального програмування

- Початківцям це непросто зрозуміти. Таким чином, це не дружній парадигмальний підхід для новачків програмістів.
- Обслуговування складно на етапі кодування, коли розмір проєкту великий.

Функція зворотного виклику

Теорія: Функціональне програмування

Функція зворотного виклику (**callback**) - це функція, що передається іншій функції як аргумент і згодом викликається всередині цієї зовнішньої функції для виконання певної операції чи дії.

Розробник, який використовує API на основі зворотних викликів, створює функцію та передає її до цього API.

Постачальник API (який виступає в ролі того, хто здійснює виклик) приймає цю функцію та викликає (або виконує) її в певний момент усередині власного коду.

Саме той, хто здійснює виклик, відповідає за передачу правильних параметрів функції зворотного виклику. Він також може очікувати від цієї функції певного значення, яке визначатиме його подальші дії.

Існує два способи виклику функції зворотного виклику: **синхронний** та **асинхронний**.

Синхронні функції зворотного виклику викликаються одразу після виклику зовнішньої функції, **без виконання проміжних асинхронних завдань**, тоді як асинхронні функції викликаються пізніше - після завершення асинхронної операції.

Виклик функції зворотного виклику

Теорія: Функціональне програмування

Розуміння того, чи викликається функція зворотного виклику синхронно, чи асинхронно, є особливо важливим при аналізі побічних ефектів. Розгляньмо такий приклад:

```
let value = 1;
doSomething(() => {
    value = 2;
});
console.log(value); // 1 or 2?
```

Якщо `doSomething` викликає функцію зворотного виклику (callback) синхронно, то остання інструкція виведе в консоль `2`, оскільки присвоєння `value = 2` виконується синхронно; в іншому разі, якщо зворотний виклик є асинхронним, остання інструкція виведе `1`, оскільки `value = 2` виконується лише після інструкції `console.log`.

Синхронні функції зворотного виклику

Теорія: Функціональне програмування

Прикладами синхронних зворотних викликів є ті, що передаються до `Array.prototype.map()`, `Array.prototype.forEach()` тощо. Приклад реалізації `doSomething`, де зворотний виклик викликається синхронно:

```
let value = 1;

// Synchronous
function doSomething(callback) {
    callback();
}

doSomething(() => {
    value = 2;
});

console.log(value); //  2
```

Асинхронні функції зворотного виклику

Теорія: Функціональне програмування

Прикладами асинхронних зворотних викликів є ті, що передаються до `setTimeout()` та `Promise.prototype.then()`. Приклад реалізації `doSomething`, де зворотний виклик викликається асинхронно:

```
let value = 1;
// Asynchronous
function doSomething(callback) {
    setTimeout(callback, 0);
}

doSomething(() => {
    value = 2;
});

console.log(value); // 1
```

Функція вищого порядку

Теорія: Функціональне програмування

У JavaScript **функція є об’єкт першого класу**.

Функції - це самодостатні модулі коду, які виконують конкретне завдання. Функції зазвичай приймають дані, обробляють їх і повертають результат. У JavaScript функції розглядаються як об’єкти першого класу. Можна розглядати функції як значення та призначати їх іншій змінній, передавати їх як аргументи іншій функції або повертати їх з іншої функції. Властивість додається до функції після того, як її було оголошено, і вона повертає значення властивості кожного разу, коли властивість викликається. Передача властивості функції можлива, але не є хорошою практикою.

Функція вищого порядку в JavaScript - це функція, яка приймає іншу функцію як аргумент або повертає функцію як результат.

Функція може отримувати іншу функцію (колбек) для виконання певної дії всередині себе.

Функція може створювати та повертати нову функцію, що корисно для замикань та каріювання.

Функція вищого порядку приймає функцію як аргумент

Теорія: Функціональне програмування

```
function operateOnArray(arr, callback) {
    const result = [];
    for (let i = 0; i < arr.length; i++) {
        result.push(callback(arr[i]));
    }
    return result;
}

const numbers = [1, 2, 3];
const doubled = operateOnArray(numbers, num => num * 2);
// [2, 4, 6]
```

Функція вищого порядку повертає функцію

Теорія: Функціональне програмуваннят

```
// Функція вищого порядку повертає функцію:

    function createMultiplier(multiplier) {
        return function (number) {
            return number * multiplier;
        };
    }

    const double = createMultiplier(2);
    console.log(double(5)); // 10
```

Функція для фільтрації парних і непарних чисел

Практика: Функціональне програмуванняї

```
// Створимо функцію для фільтрації парних і непарних чисел:
function filterFunction(arr, callback) {
    const filteredArr = [];
    for (let i = 0; i < arr.length; i++) { callback(arr[i]) ? filteredArr.push(arr[i]) : null; }
    return filteredArr;
}

const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11];
function isOdd(x) { return x % 2 != 0; } // Функція для фільтрації непарних чисел
function isEven(x) { return x % 2 === 0; } // Функція для фільтрації парних чисел
console.log(filterFunction(arr, isOdd)) // [ 1, 3, 5, 7, 9, 11 ] // For filtering out odd numbers
console.log(filterFunction(arr, isEven)) // [ 2, 4, 6, 8, 10 ] // For filtering out even numbers

// можемо передати будь-яку іншу логіку фільтрації filterFunction, без необхідності її змінювати:
function isGreaterThanFive(x) { return x > 5; }
console.log(filterFunction(arr, isGreaterThanFive)) // [ 6, 7, 8, 9, 10, 11 ]
// або
console.log(filterFunction(arr, (x) => x > 5)) // [ 6, 7, 8, 9, 10, 11 ]
```

Вбудовані методи масивів

Теорія: Функціональне програмування

У JavaScript найчастіше використовуються вбудовані функції вищого порядку, такі як map(), filter() та reduce().

Для фільтрації даних за категоріями в JavaScript найчастіше використовується вбудований метод масивів **filter**().

Метод **map**() у JavaScript - це функція вищого порядку, яка створює новий масив, застосовуючи передану функцію до кожного елемента старого масиву.\

Метод reduce() в JavaScript - це функція вищого порядку, яка послідовно застосовує вказану функцію до кожного елемента масиву, зводячи його до одного єдиного значення.

Метод filter()

Теорія: Функціональне програмування

Метод filter() в JavaScript створює новий масив з елементами старого масиву, які пройшли перевірку в переданій функції.

Метод filter не змінює старий масив: оригінальний масив залишається без змін.

Повертає новий масив: містить лише ті елементи, для яких умова повернула true. Якщо жоден елемент не підійшов, повертається порожній масив [].

Синтаксис

```
const newArray = array.filter(function(element, index, array) {
  // умова повертає true або false
});
```

element - поточний елемент масиву, який зараз обробляється.

index (необов'язково) - індекс цього елемента.

array (необов'язково) - сам масив, над яким виконується метод.

Метод map()

Теорія: Функціональне програмування

Метод map() у JavaScript - це функція вищого порядку, яка створює новий масив, застосовуючи передану функцію до кожного елемента старого масиву.

Проходить по кожному елементу масиву, змінює його за допомогою вашого правила і повертає новий масив, не змінюючи старий.

Це функція вищого порядку, тому що вона приймає іншу функцію (колбек) як аргумент.

Функція всередині map зазвичай отримує три значення: поточний елемент, індекс (номер елемента) та сам масив.

```
const numbers = [1, 2, 3];
// Множимо кожен елемент на 2
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6]
console.log(numbers); // [1, 2, 3] (старий масив не змінився)
```

Метод reduce()

Теорія: Функціональне програмування

Метод reduce() в JavaScript - це функція вищого порядку, яка послідовно застосовує вказану функцію до кожного елемента масиву, зводячи його до одного єдиного значення.

Синтаксис

array.reduce(callback(accumulator, currentValue, index, array), initialValue)

accumulator (акумулятор) — значення, яке накопичується після кожного кроку (результат попереднього виклику функції).

currentValue (поточний елемент) — поточний елемент масиву, що обробляється.

initialValue (початкове значення) — значення, з якого починається акумулятор на першому кроці.

Приклади reduce()

Теорія: Функціональне програмування

```
// Сума елементів масиву:j
    const numbers = [1, 2, 3, 4];
    const sum = numbers.reduce((acc, curr) => acc + curr, 0);
    // Результат: 10

 // Знаходження максимального числа:
    const numbers = [5, 12, 8, 130, 44];
    const max = numbers.reduce((acc, curr) => curr > acc ? curr : acc, numbers[0]);
    // Результат: 13
```

Замикання

Робота з функціями map, filter та reduce

Проста фільтрація за однією категорією

Практика: Функціональне програмування

```
// Початковий масив даних
const products = [
 { id: 1, name: 'Ноутбук', category: 'Електроніка' },
 { id: 2, name: 'Футболка', category: 'Одяг' },
 { id: 3, name: 'Телефон', category: 'Електроніка' },
 { id: 4, name: 'Джинси', category: 'Одяг' }
];

// Функція для фільтрації
function filterByCategory(items, categoryName) {
 return items.filter(item => item.category === categoryName);
}

// Використання
const electronics = filterByCategory(products, 'Електроніка');
console.log(electronics);
// Виведе: Ноутбук та Телефон
```

Множинна фільтрація за кількома категоріями

Практика: Функціональне програмування

Якщо користувач вибирає кілька категорій одночасно (наприклад, через чекбокси), використовується метод includes() всередині filter():

```
const products = [
 { id: 1, name: 'Ноутбук', category: 'Електроніка' },
 { id: 2, name: 'Футболка', category: 'Одяг' },
 { id: 3, name: 'Книга', category: 'Книги' },
 { id: 4, name: 'Джинси', category: 'Одяг' }
];

// Активні фільтри (вибрані користувачем)
const selectedCategories = ['Електроніка', 'Книги'];
// Фільтрація
const filteredProducts = products.filter(product =>
 selectedCategories.includes(product.category)
);
console.log(filteredProducts);
// Виведе: Ноутбук та Книга
```

Підключення фільтрації до HTML-елементів

Практика: Функціональне програмування

Програма буде автоматично реагувати на кожну зміну стану чекбоксів (подія change), оновлювати список активних категорій та заново перерендерювати інтерфейс.

1. Динамічний збір фільтрів: Щоразу, коли користувач ставить або знімає галочку, спрацьовує метод handleFilter. Через Array.from(checkboxes).filter(...) ми миттєво отримуємо актуальний масив рядків (наприклад, ['Електроніка', 'Одяг']).
\
2. Умова "порожнього фільтра": Рядок if (activeCategories.length === 0) return true; забезпечує базову поведінку інтернет-магазинів: якщо користувач ще нічого не вибрав, показується весь асортимент.

3. Чистий рендер: Функція renderProducts повністю видаляє старий HTML-вміст контейнера за допомогою container.innerHTML = '' і створює нові картки на основі відфільтрованого масиву.

Макет

Практика: Функціональне програмування

```
<!-- Бокова панель з чекбоксами -->
  <aside class="sidebar">
     <h3>Категорії</h3>
     <div class="checkbox-group">
        <label><input type="checkbox" value="Електроніка" class="category-checkbox">
Електроніка</label>
        <label><input type="checkbox" value="Одяг" class="category-checkbox"> Одяг</label>
        <label><input type="checkbox" value="Книги" class="category-checkbox"> Книги</label>
     </div>
  </aside>
  <!-- Контейнер для виведення результатів -->
  <main class="content">
     <h2>Товари</h2>
     <div id="productsContainer" class="products-list"></div>
  </main>
  <script src="catalog.js"></script>
```

Стилі макету

Практика: Функціональне програмування

```
body {
    font-family: sans-serif;
    display: flex;
    gap: 20px;
    padding: 20px;
}

.sidebar {
    width: 200px;
    padding: 15px;
    border: 1px solid #ccc;
    border-radius: 8px;
}

.sidebar h3 {
    margin-top: 0;
}
```

Стилі прапорців

Практика: Функціональне програмування

```
.checkbox-group {
    display: flex;
    flex-direction: column;
    gap: 10px;
}

.content {
    flex-grow: 1;
}
```

Стилі картки товару

Практика: Функціональне програмування

```css
.products-list {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
    gap: 15px;
}

.product-card {
    padding: 15px;
    border: 1px solid #ddd;
    border-radius: 8px;
    box-shadow: 2px 2px 5px rgba(0, 0, 0, 0.05);
}

.product-category {
    color: #666;
    font-size: 0.9em;
}
```

Вхідні дані

Практика: Функціональне програмування

```
// 1. Вхідні дані (масив об'єктів)
const products = [
 { id: 1, name: 'Ноутбук', category: 'Електроніка' },
 { id: 2, name: 'Футболка', category: 'Одяг' },
 { id: 3, name: 'Книга "JS для початківців"', category: 'Книги' },
 { id: 4, name: 'Телефон', category: 'Електроніка' },
 { id: 5, name: 'Джинси', category: 'Одяг' },
 { id: 6, name: 'Електронна книга', category: 'Електроніка' }
];
```

Пошук DOM-елементів

Практика: Функціональне програмування

```
// 2. Пошук DOM-елементів
const container = document.getElementById('productsContainer');
const checkboxes = document.querySelectorAll('.category-checkbox');

// 3. Функція для відображення товарів на сторінці
function renderProducts(items) {
 // Очищуємо контейнер перед новим рендером
 container.innerHTML = '';

 // Якщо нічого не знайдено за фільтрами
 if (items.length === 0) {
   container.innerHTML = '<p>Товарів не знайдено</p>';
   return;
 }
```

Створюємо HTML-картку для кожного об'єкта

Практика: Функціональне програмування

```
// Створюємо HTML-картку для кожного об'єкта
 items.forEach(product => {
   const card = document.createElement('div');
   card.className = 'product-card';
   card.innerHTML = `
     <h4>${product.name}</h4>
     <p class="product-category">Категорія: ${product.category}</p>
   `;
   container.appendChild(card);
 });
}
```

Головна функція фільтрації

Практика: Функціональне програмування

```
// 4. Головна функція фільтрації
function handleFilter() {
 // Збираємо значення лише тих чекбоксів, які зараз відмічені (checked)
 const activeCategories = Array.from(checkboxes)
   .filter(checkbox => checkbox.checked)
   .map(checkbox => checkbox.value);

 // Логіка фільтрації:
 // Якщо жоден чекбокс не вибрано — показуємо ВСІ товари.
 // Якщо вибрано — залишаємо лише ті товари, чия категорія є в масиві activeCategories.
 const filteredProducts = products.filter(product => {
   if (activeCategories.length === 0) return true;
   return activeCategories.includes(product.category);
 });

 // Перемальовуємо інтерфейс з відфільтрованими даними
 renderProducts(filteredProducts);
}
```

Слухач подій change

Практика: Функціональне програмування

```
// 5. Навішуємо слухач подій на кожен чекбокс
checkboxes.forEach(checkbox => {
 checkbox.addEventListener('change', handleFilter);
});


// Первинний запуск програми, щоб відобразити всі товари під час завантаження сторінки
renderProducts(products);
```

Комплексний фільтр товарів

Практика: Функціональне програмування

Для об'єднання кількох фільтрів разом із сортуванням, найкраще використовувати єдину функцію фільтрації.

Вона буде запускатися при будь-якій зміні вхідних параметрів та послідовно застосовувати всі правила до початкового масиву даних.

Макет

Практика: Функціональне програмування

```
<!-- Бокова панель фільтрів -->
  <aside class="sidebar">
      <!-- Пошук за назвою -->
      <div class="filter-group">
         <h3>Пошук</h3>
         <input type="text" id="searchInput" class="search-input" placeholder="Введіть назву...">
      </div>

      <!-- Фільтр за категоріями -->
      <div class="filter-group">
         <h3>Категорії</h3>
         <label><input type="checkbox" value="Електроніка" class="category-checkbox">
Електроніка</label>
         <label><input type="checkbox" value="Одяг" class="category-checkbox"> Одяг</label>
         <label><input type="checkbox" value="Книги" class="category-checkbox"> Книги</label>
      </div>
```

Макет

Практика: Функціональне програмування

```
<!-- Фільтр за ціною -->
<div class="filter-group">
    <h3>Ціна (грн)</h3>
    <div class="price-inputs">
        <input type="number" id="priceMin" placeholder="Від" min="0">
        <input type="number" id="priceMax" placeholder="До" min="0">
    </div>
</div>
```

Макет

Практика: Функціональне програмування

```
    <!-- Сортування -->
    <div class="filter-group">
        <h3>Сортування</h3>
        <select id="sortSelect" class="sort-select">
            <option value="default">За замовчуванням</option>
            <option value="price-asc">Від дешевих до дорогих</option>
            <option value="price-desc">Від дорогих до дешевих</option>
            <option value="name-asc">За назвою (А-Я)</option>
        </select>
    </div>
</aside>

<!-- Контентна частина -->
<main class="content">
    <h2>Товари</h2>
    <div id="productsContainer" class="products-list"></div>
</main>
```

Стилі макету

Практика: Функціональне програмування

```
body {
    font-family: sans-serif;
    display: flex;
    gap: 20px;
    padding: 20px;
    margin: 0;
    background: #f9f9f9;
}
.sidebar {
    width: 260px;
    padding: 20px;
    border: 1px solid #ddd;
    border-radius: 8px;
    background: #fff;
    display: flex;
    flex-direction: column;
    gap: 20px;
}
```

Стилі макету

Практика: Функціональне програмування

```
.sidebar h3 {
    margin: 0 0 10px 0;
    font-size: 1.1em;
    border-bottom: 1px solid #eee;
    padding-bottom: 5px;
}

.filter-group {
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.price-inputs {
    display: flex;
    gap: 10px;
}
```

Стилі макету

Практика: Функціональне програмування

```
.price-inputs input {
    width: 100%;
    padding: 5px;
}
.search-input,
.sort-select {
    padding: 8px;
    width: 100%;
    box-sizing: border-box;
}
.content {
    flex-grow: 1;
}
.products-list {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
    gap: 15px;
}
```

Стилі макету

Практика: Функціональне програмування

```
.product-card {
    padding: 15px;
    border: 1px solid #ddd;
    border-radius: 8px;
    background: #fff;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.02);
}

.product-price {
    font-weight: bold;
    color: #2c3e50;
    margin: 8px 0;
}
.product-category {
    color: #888;
    font-size: 0.85em;
}
```

Розширений масив об'єктів

Практика: Функціональне програмування

```
// 1. Розширений масив об'єктів (додано поле price)

const products = [
  { id: 1, name: 'Ноутбук Lenovo', category: 'Електроніка', price: 25000 },
  { id: 2, name: 'Футболка літня', category: 'Одяг', price: 450 },
  { id: 3, name: 'Книга "JS для початківців"', category: 'Книги', price: 350 },
  { id: 4, name: 'Телефон Samsung', category: 'Електроніка', price: 12000 },
  { id: 5, name: 'Джинси класичні', category: 'Одяг', price: 1200 },
  { id: 6, name: 'Електронна книга PocketBook', category: 'Електроніка', price: 4500 },
  { id: 7, name: 'Книга "Алгоритми"', category: 'Книги', price: 600 }
];
```

DOM Елементи

Практика: Функціональне програмування

```
// 2. DOM Елементи

const container = document.getElementById('productsContainer');
const searchInput = document.getElementById('searchInput');
const checkboxes = document.querySelectorAll('.category-checkbox');
const priceMinInput = document.getElementById('priceMin');
const priceMaxInput = document.getElementById('priceMax');
const sortSelect = document.getElementById('sortSelect');
```

Функція рендеру карток у DOM

Практика: Функціональне програмування

```
// 3. Функція рендеру карток у DOM
function renderProducts(items) {
  container.innerHTML = '';
  if (items.length === 0) {
    container.innerHTML = '<p>Товарів не знайдено за вказаними фільтрами.</p>';
    return;
  }

  items.forEach(product => {
    const card = document.createElement('div');
    card.className = 'product-card';
    card.innerHTML = `
   <h4>${product.name}</h4>
   <p class="product-price">${product.price} грн</p>
   <p class="product-category">Категорія: ${product.category}</p>
  `;
    container.appendChild(card);
  });
```

Фільтрація + Сортування

Практика: Функціональне програмування

```
// 4. Головна комбінована функція: Фільтрація + Сортування
function updateInterface() {
    // --- Крок А: Отримання поточних значень з DOM ---
    const searchQuery = searchInput.value.toLowerCase().trim();

    const activeCategories = Array.from(checkboxes)
        .filter(cb => cb.checked)
        .map(cb => cb.value);

    const minPrice = parseFloat(priceMinInput.value) || 0;
    const maxPrice = parseFloat(priceMaxInput.value) || Infinity;
    const sortType = sortSelect.value;
```

Послідовна фільтрація масиву

Практика: Функціональне програмування

```
// --- Крок B: Послідовна фільтрація масиву ---
let result = products.filter(product => {
   // 1. Фільтр за назвою (регістронезалежний)
   const matchesSearch = product.name.toLowerCase().includes(searchQuery);

   // 2. Фільтр за категоріями
   const matchesCategory = activeCategories.length === 0 || activeCategories.includes(product.category);

   // 3. Фільтр за ціновим діапазоном
   const matchesPrice = product.price >= minPrice && product.price <= maxPrice;

   // Елемент залишається, лише якщо пройшов ВСІ 3 перевірки
   return matchesSearch && matchesCategory && matchesPrice;
});
```

Сортування результату

Практика: Функціональне програмування

```
// --- Крок C: Сортування результату ---
if (sortType === 'price-asc') {
   result.sort((a, b) => a.price - b.price);
} else if (sortType === 'price-desc') {
   result.sort((a, b) => b.price - a.price);
} else if (sortType === 'name-asc') {
   result.sort((a, b) => a.name.localeCompare(b.name, 'uk'));
}

// --- Крок D: Виведення результату ---
renderProducts(result);
}
```

Прив'язка подій

Практика: Функціональне програмування

```
// 5. Прив'язка подій (Event Listeners)
searchInput.addEventListener('input', updateInterface);
priceMinInput.addEventListener('input', updateInterface);
priceMaxInput.addEventListener('input', updateInterface);
sortSelect.addEventListener('change', updateInterface);

checkboxes.forEach(cb => {
   cb.addEventListener('change', updateInterface);
});

// Перший запуск для показу всіх товарів
updateInterface();
```

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua